

实验5实验报告

实验内容

1. 只读存储器实验

实验内容

实验要求根据输入的 ASCII 码值，在 LED 点阵中显示该 ASCII 码的字形；如果 ASCII 值不在可显示字符范围内，则输出 ASCII 码错误标志位。

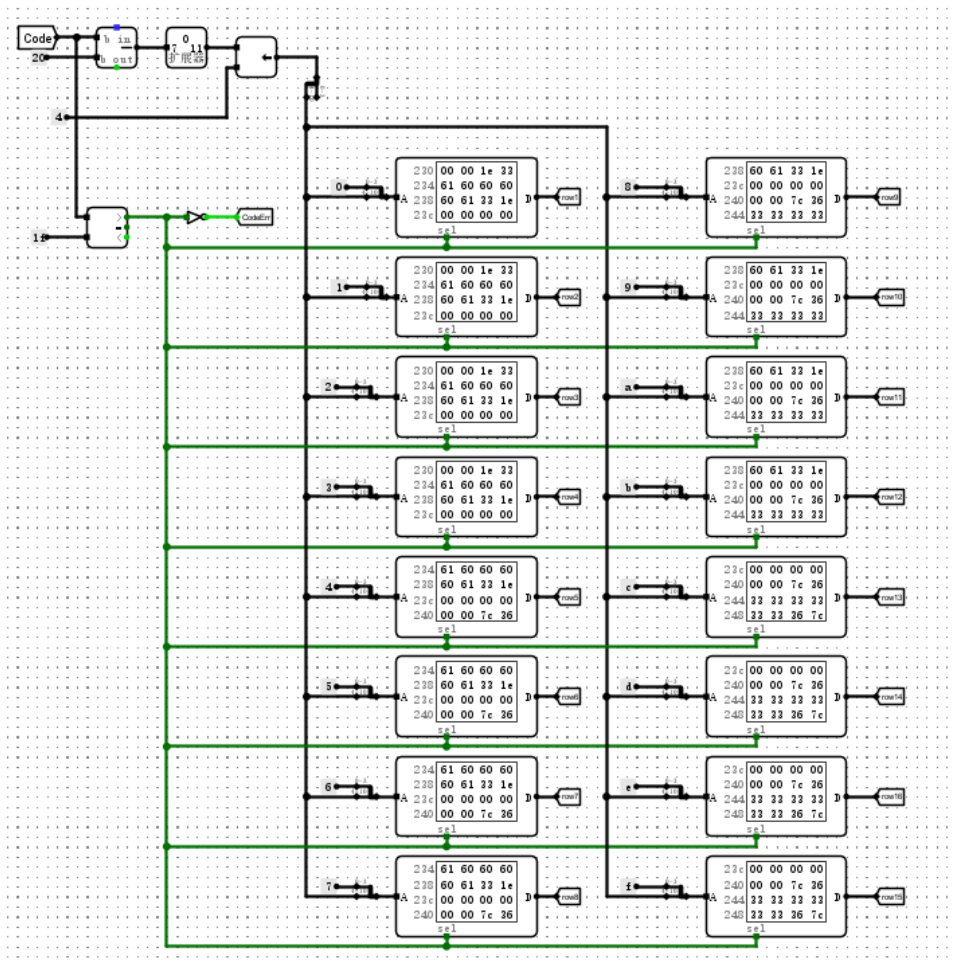
整体方案设计

输入输出引脚：

- code：输入引脚，代表输入的ascii码值
- row1-row16：对应的字符点阵数据，一个字符点阵数据为连续存放的16个字节。
- codeErr：检查输入的ascii值是否在可显示字符范围内

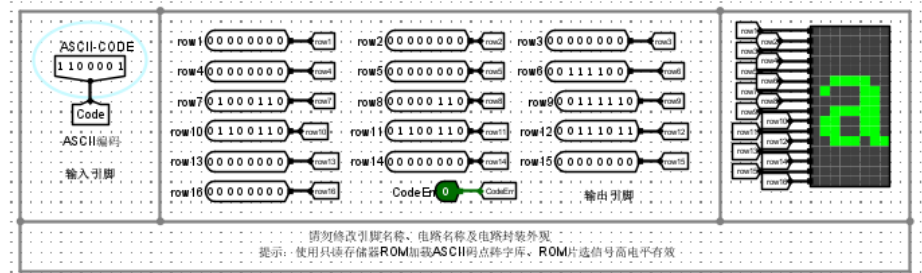
原理图和电路图

实验电路图如下：

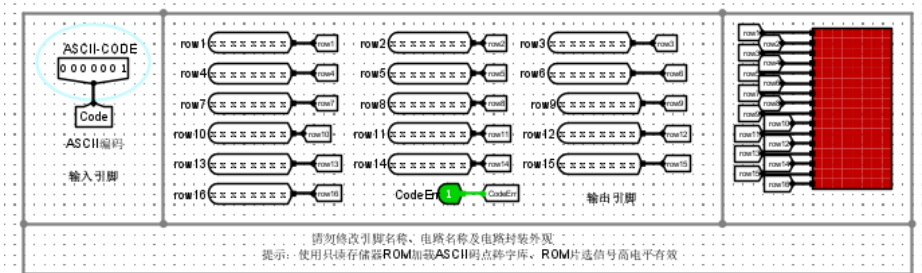


仿真测试图

令code = 1100001,对应的ascii码为a,正确输出



令code = 0000001,则输出ascii码错误标志位，与预期符合



错误分析

在完成实验的过程中，没有遇到任何错误。

2. 数据存储器实验

实验内容

设计一个256KB的数据存储器，可按照字节进行存取操作，数据字长为32位，则地址位为18位。

整体方案设计

输入引脚：

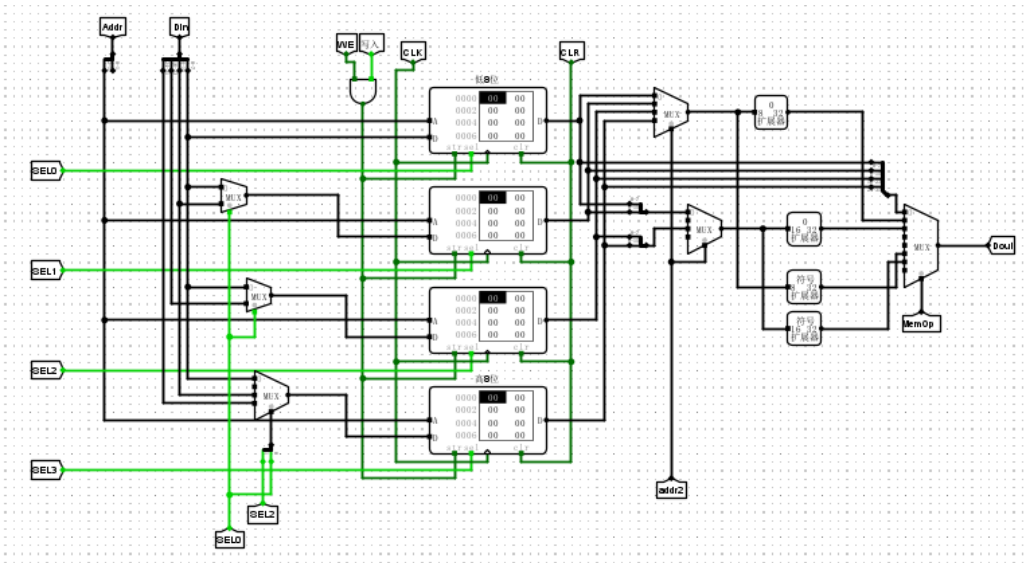
- addr：要访问的数据的地址
- din：要写入的数据
- clk：时钟信号
- clr：清空信号
- we：写使能信号，只有为1时才可能写入
- Memop：表示当前指令读写数据的字节长度

输出引脚：

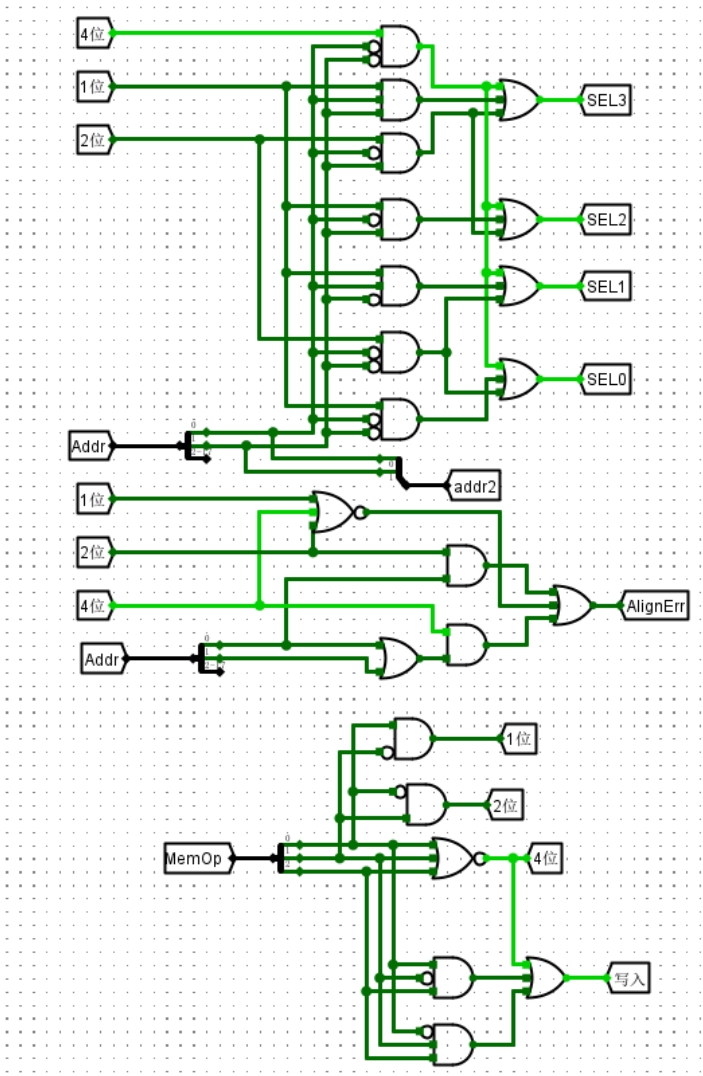
- dout：输入的读取数据值
- Alignerr：当存取多字节数据的有效地址不是自然对齐时，或memop出现未定义信号时，Alignerr=1

原理图和电路图

主电路图如图所示：

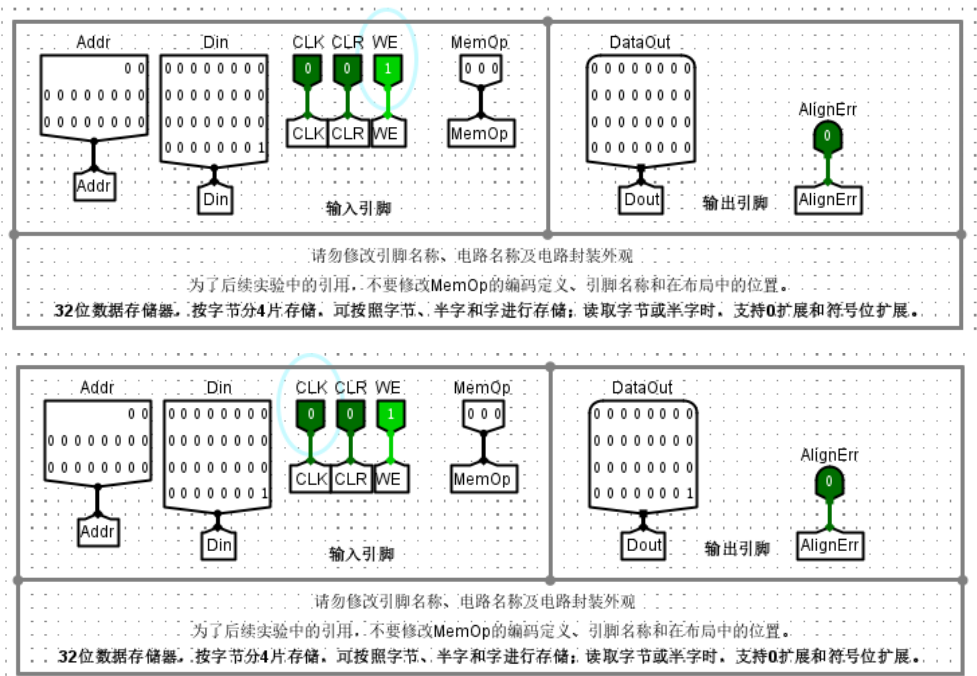


根据MemOp控制信号和地址的低 2 位选中合适的 RAM 组件进行数据的存储以及如何组织输出数据的电路图如下：



仿真测试图

第一个时钟周期：



第二个时钟周期：

错误分析

在完成实验的过程中，没有遇到任何错误。

3. 取指令部件IFU实验

实验内容

实现一个取指令部件，可以根据初始地址 Initial Address、立即数寄存器 Imm和 rs1寄存器的数据 BusA，以及控制信号Reset、NxtASrc和NxtBSrc的赋值输出当前指令的地址寄存器PC和指令存储器的输出内容IR。

整体方案设计

输入引脚：

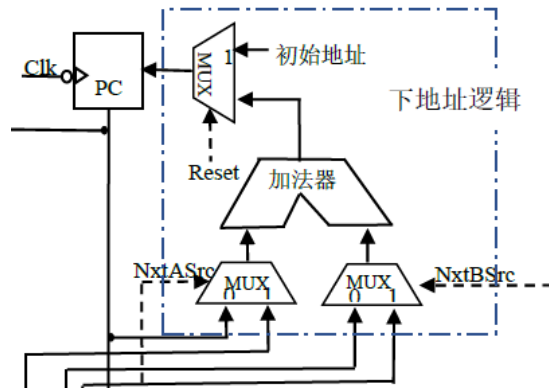
- InitAddr：设定的初始地址值
- Imm：立即数值，用来模拟jal或jalr的跳转
- BusA：从内存中读取出的值，即jalr中的R[rs1]
- CLK：时钟信号
- Reset：重置信号，当它为1时，在下一个时钟信号有效后，将把初始地址Initial Address加载到程序计数器PC中
- NPCASrc：当NPCASrc=0时，选择PC寄存器的值，否则选择Rs1寄存器值BusA
- NPCBSrc：当NPCBSrc=0时选择常量4，否则选择立即数Imm。

输出引脚：

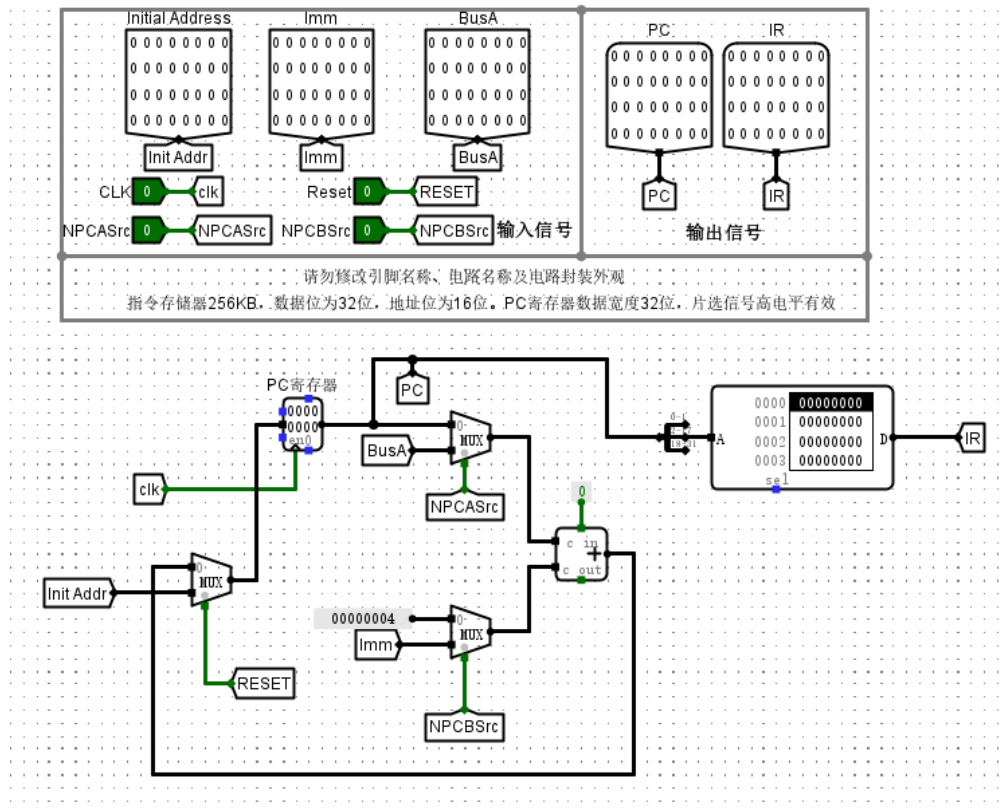
- PC：当前指令的地址寄存器PC
- IR：指令存储器的输出内容IR

原理图和电路图

实验原理图如下，本质上IFU就是实现PC的更新，并输出下一条指令：

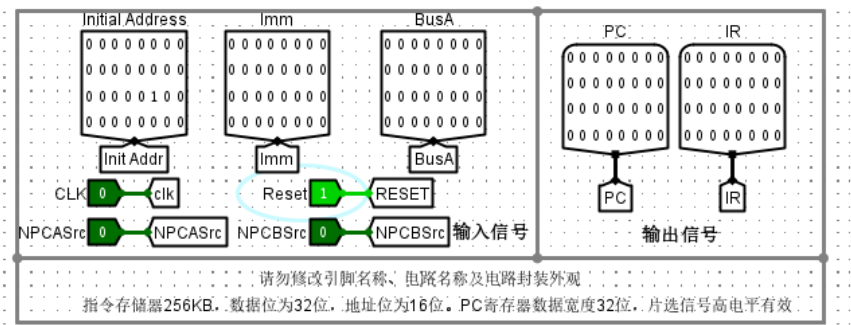


实验电路图如下:

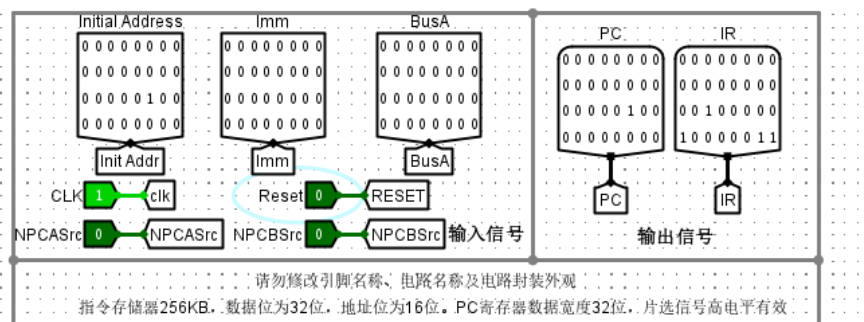


仿真测试图

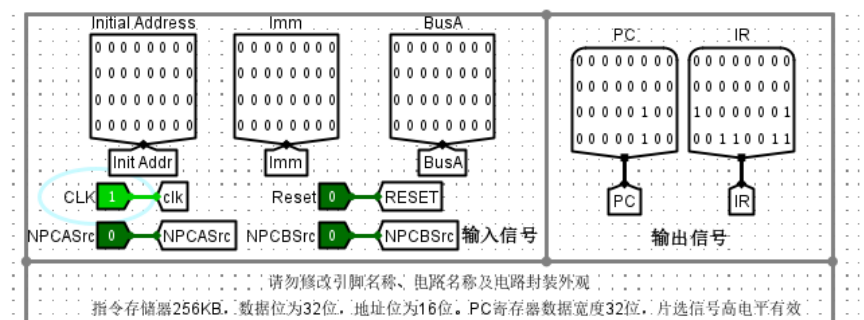
初始状态:



第一个时钟周期:



第二个时钟周期:



错误分析

在完成实验的过程中, 没有遇到任何错误。

4. 取操作数部件IDU实验

实验内容

设计一个取操作数部件IDU, 实现立即数拓展, 从指令中读取操作码, 操作数等的功能。

整体方案设计

输入引脚:

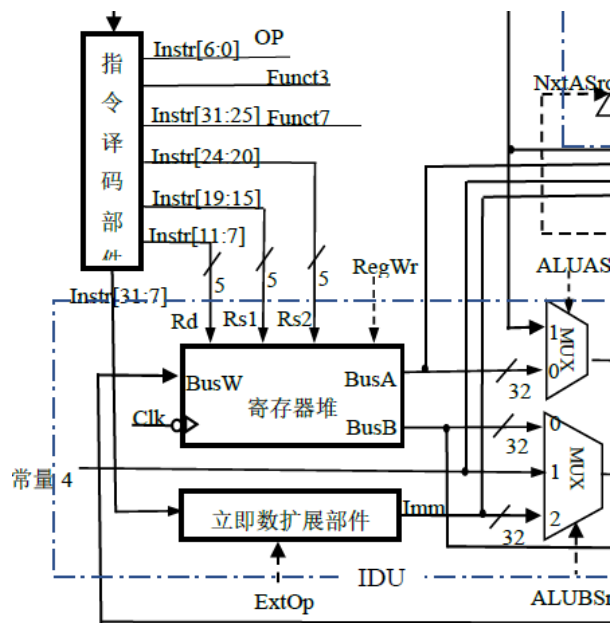
- IR: 送入的指令
- PC: 程序计数器当前的值
- BusW: 要写入寄存器堆的值
- ExtOp: 指令的类型, 根据ExtOp的不同选择不同的立即数拓展方式
- RegWr: 寄存器堆写入使能端, 为1代表允许写入
- ALUASrc: 选择ALU的操作数A, 当ALUASsrc=0时输出BusA, 当ALUASsrc=1时输出PC。
- ALUBsrc: 选择ALU的操作数B, 当ALUBSsrc=00时输出BusB, 当ALUBSsrc=01时选择输出常数4, 当ALUBSsrc=10时选择输出32位立即数mm。
- CLK: 时钟信号

输出引脚:

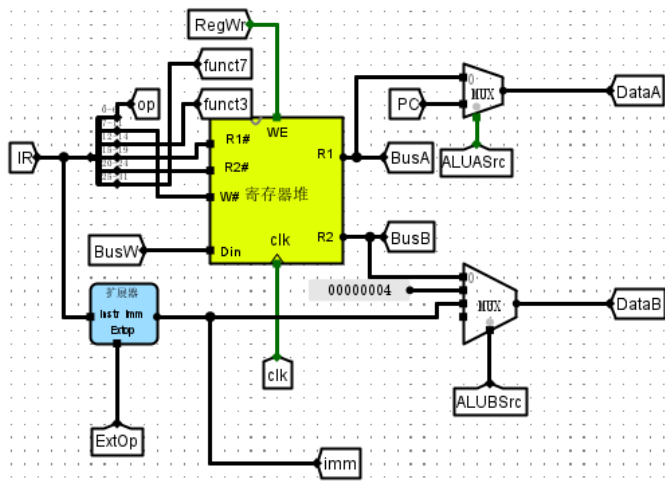
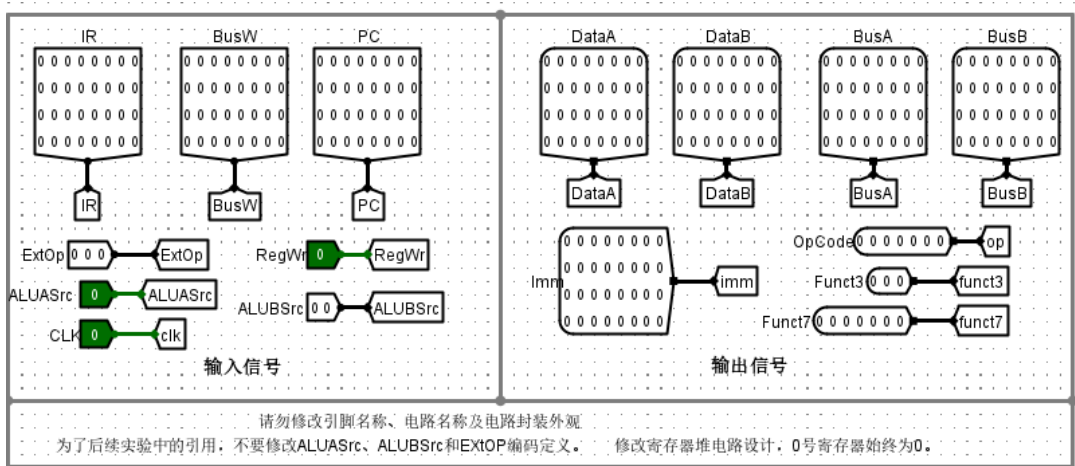
- DataA,B: 输出到ALU的两个值, 代表交给ALU的两个操作数
- BusA,B: 寄存器堆输出的两个值, 代表从寄存器堆直接读出的两个值。
- Imm: 经过拓展的立即数值
- Opcode, func3, func7: 指令的操作码, func3码, func7码, 用来说明指令的功能

原理图和电路图

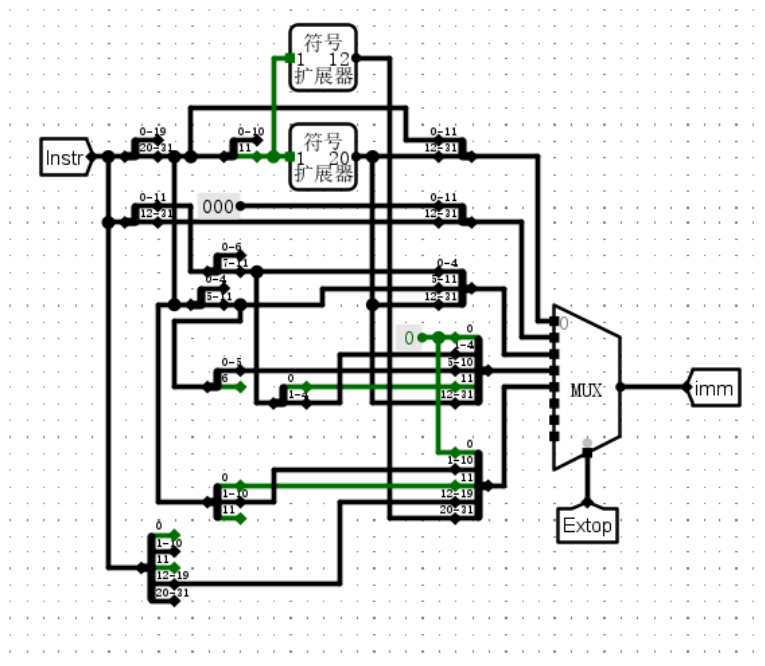
原理图如下：



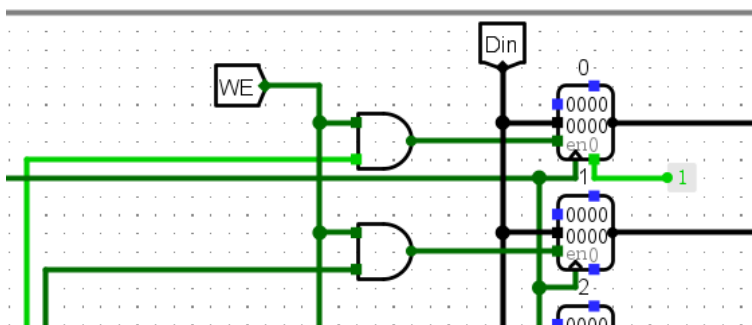
主电路图如图：



立即数拓展器电路图如图：



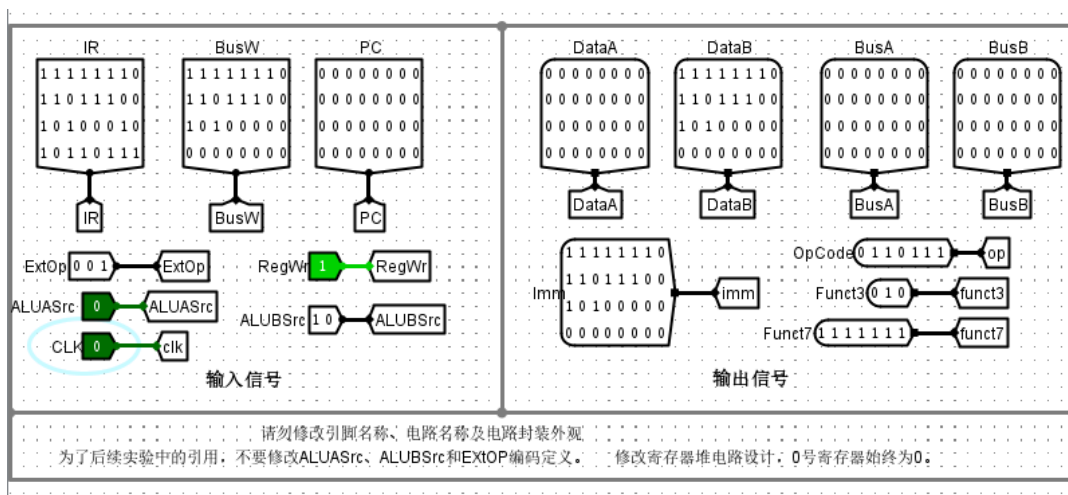
在regfile中，对0号寄存器做修改，使其始终为0



仿真测试图

输入：IR = fedca2b7, BusW = fedca000, PC = 00000000, ExtOp = 1, RegWr = 1, ALUASrc = 0, ALUBSrc = 2

输出结果符合预期。



错误分析

在完成实验的过程中，没有遇到任何错误。

5. 数据通路实验

实验内容

数据通路是具体完成数据存取、运算的部件。单周期CPU的数据通道是指获取到指令之后，根据指令内容，读取操作数，进行操作，得到结果并写回的过程。本实验要求在前面几个实验的基础上，设计一个数据通路

整体方案设计

输入引脚：

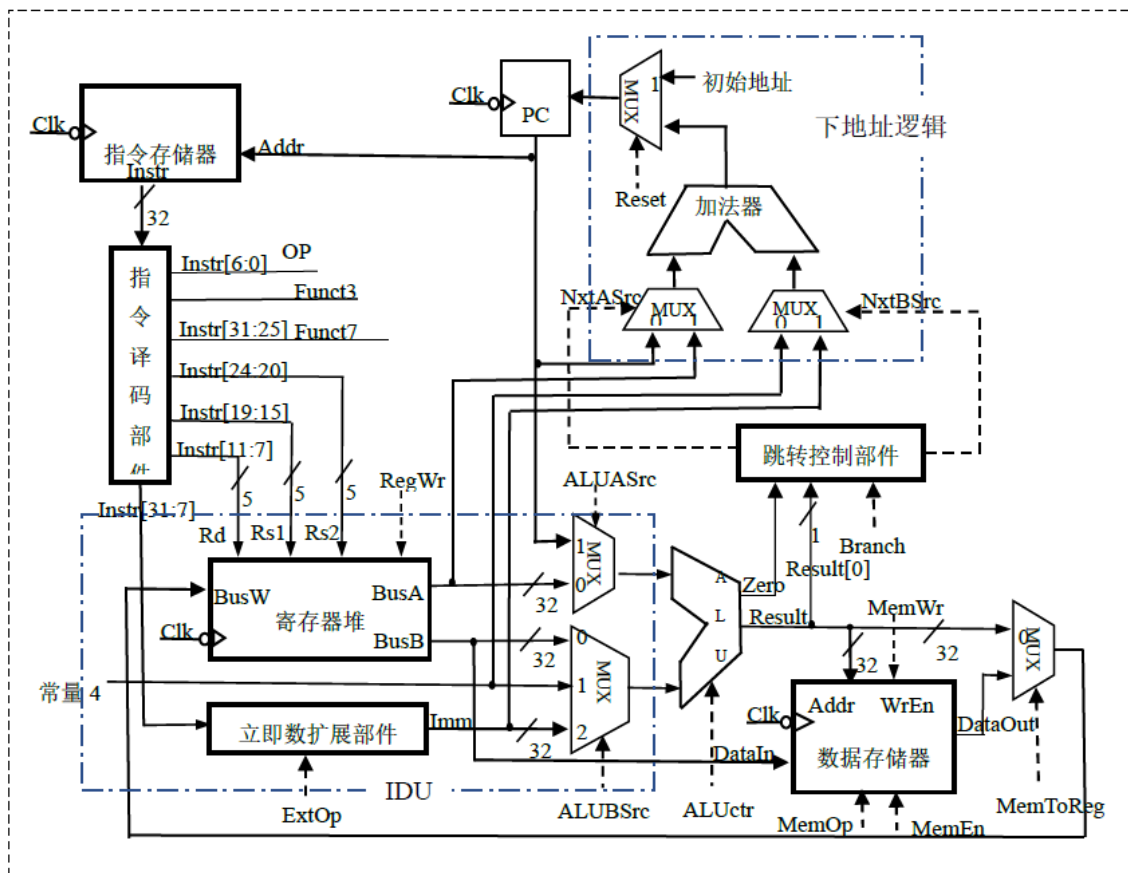
- InitialAddress：要载入的初始地址
- ReSet：重置信号，当它为1时，在下一个时钟信号有效后，将把初始地址Initial Address加载到程序计数器PC中
- ExtOp：指令的类型，根据ExtOp的不同选择不同的立即数拓展方式
- Memop：表示当前指令读写数据的字节长度
- RegWr：寄存器堆写入使能端，为1代表允许写入
- ALUASr、ALUBSr：ALU的两个操作数
- ALUctr：控制ALU执行什么运算
- MemWr：数据存储器写入使能端
- MemToR：为1表示选择数据存储器的输出作为BusW，即作为要写入寄存器堆的值
- Branch：表示不同类型的跳转指令

输出引脚：

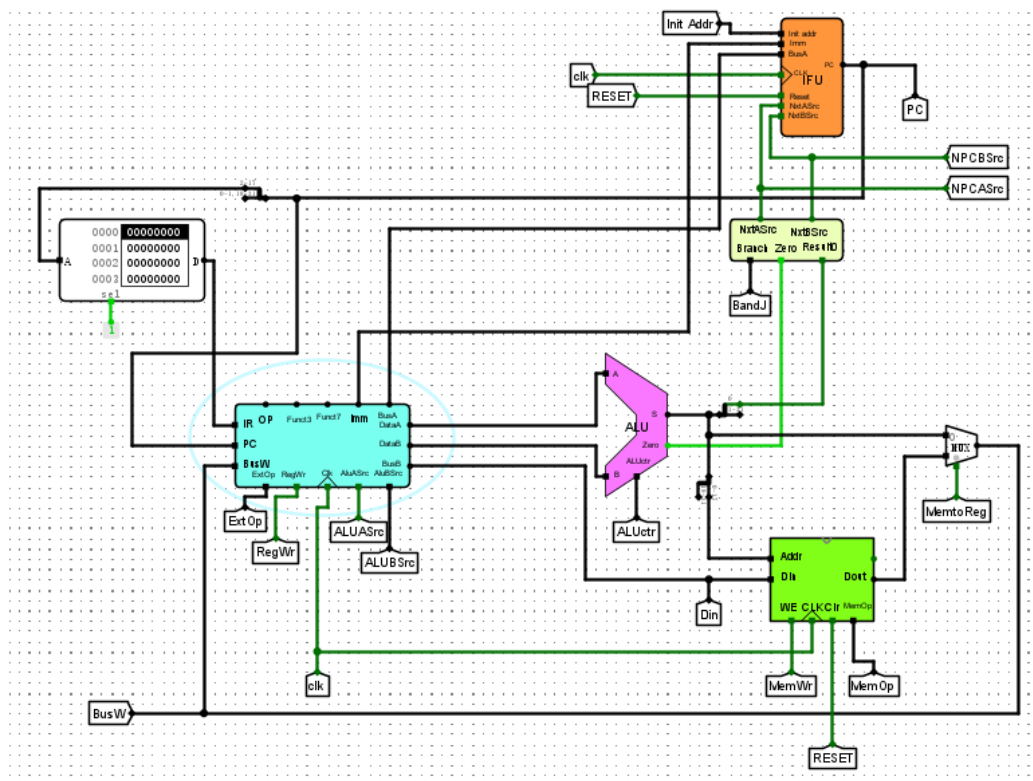
- PC：当前程序计数器的值
- BusW：总线写入值，也就是要写入寄存器堆的值
- Din：输入数据存储器的值，也是寄存器堆输出的BusB，即要写入数据存储器的值。
- NPCASrc：当NPCASrc=0时，选择PC寄存器的值，否则选择Rs1寄存器值BusA
- NPCBsrc：当NPCBsrc=0时选择常量4，否则选择立即数Imm。

原理图和电路图

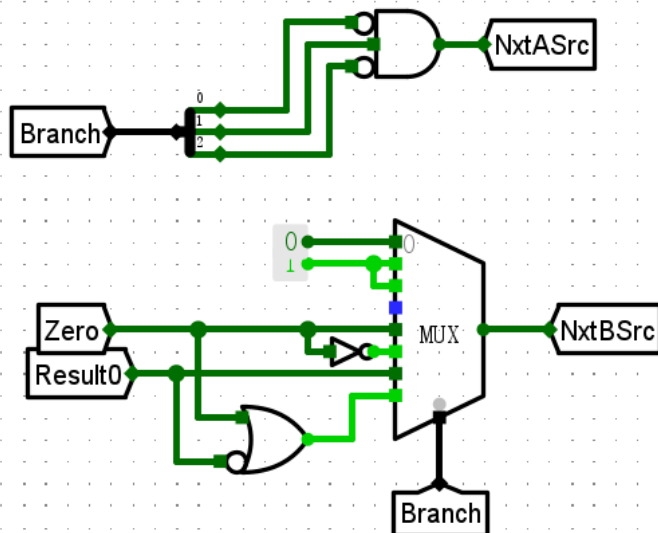
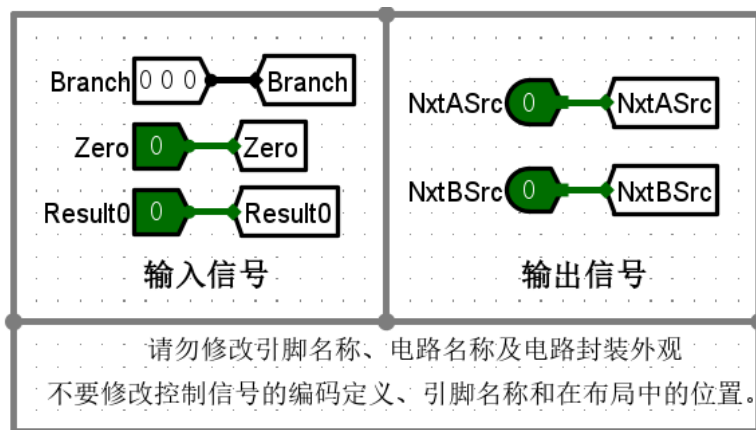
实验原理图如下：



实验电路图如下：

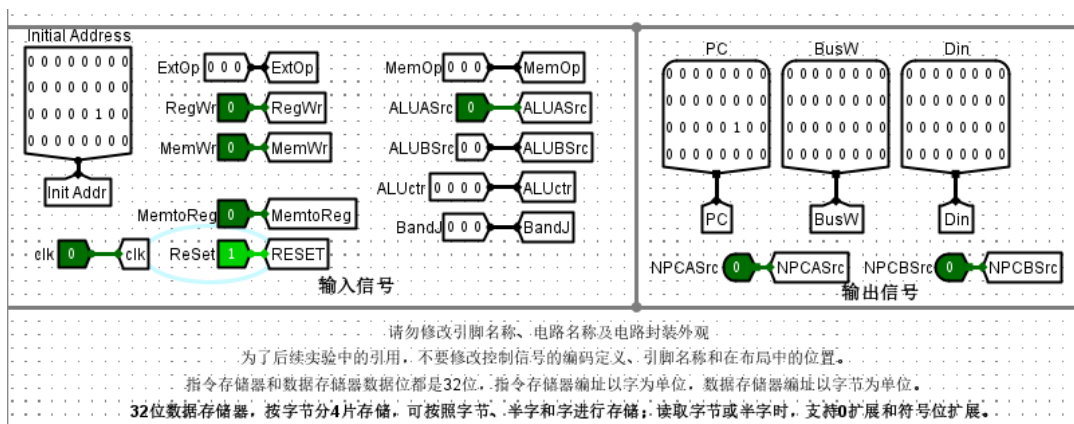


跳转控制部件的电路图如下：

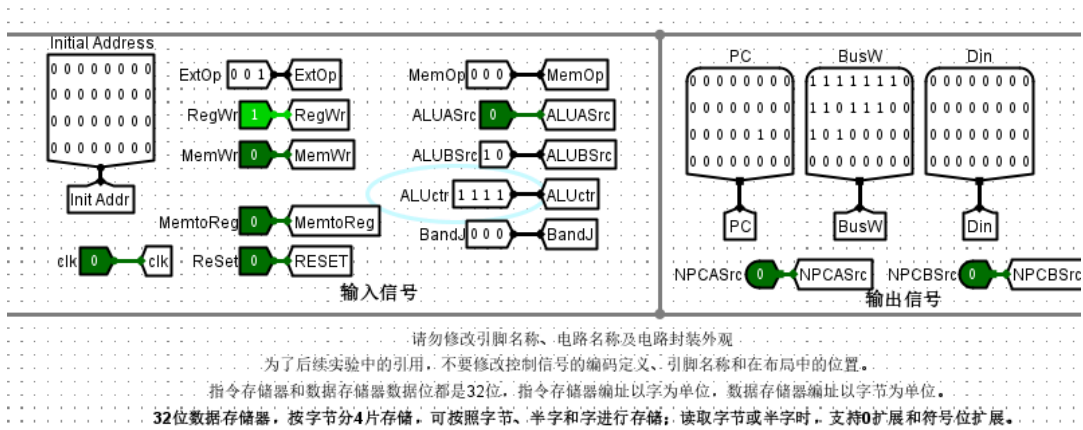


仿真测试图

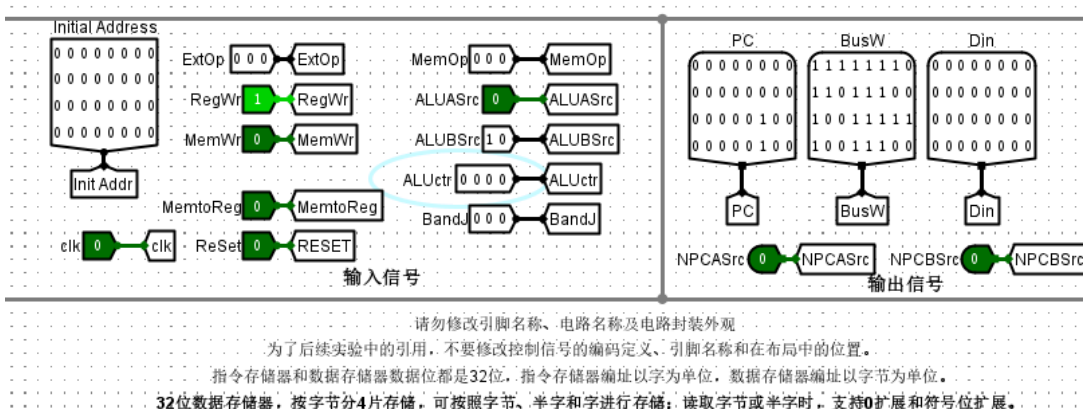
加载的测试：



完成加载后，进行执行，取ExtOp=1,RegWr=1,ALUBSrc=2,ALUctr=f,其他值为0，得到BusW=fedca000，符合预期



一个时钟周期后，再令RegWr=1,ALUBSrc=2，其他值为0，得BusW=fedc9f9c，符合预期



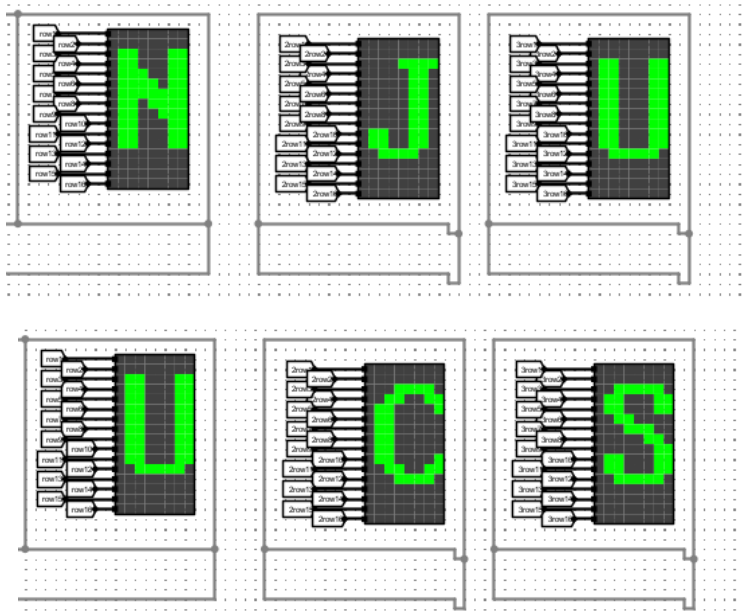
错误分析

在完成实验的过程中，没有遇到任何错误。

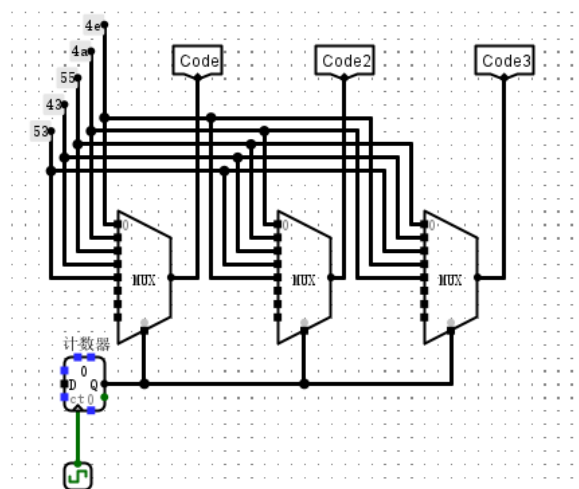
思考题

T1

变化图：



电路图：



实现思路为，分别将NJUCS代表的字符数据输入不同的多路选择器中，用计数器控制每个时钟周期三个LED屏上所显示的值即可。

T2

会引发数据同步问题，比如说，由于PC寄存器和数据存储器写入时是上升沿触发有效，而寄存器堆，即IDU是下降沿触发有效，假设在同一个时钟周期内需要读取PC寄存器的值并将其写入寄存器堆或数据存储器，则在同一个时钟周期内可能会发生读取到的PC值并未及时反映到寄存器堆的情况。再比如说目前程序要实现一个循环，PC值在几个时钟周期内会在某几个值之间循环，则由于时钟未同步，可能打破这个循环，使程序发生不正确的行为。

T3

1. 使用无操作 (NOP) 指令

在程序结束时，通过插入无操作指令（NOP，No Operation）来使CPU暂停执行一段时间。这并不会完全停止CPU的工作，但可以让CPU“空转”，直到手动干预或外部条件触发。

2. 触发硬件中断

当CPU读取并执行完最后一条指令之后，发出一个特殊信号，触发硬件中断，让CPU暂时中止当前程序的执行，并进入中断处理程序。在中断处理程序中，系统可以执行关闭操作或者进入待命状态。通过这种方式，CPU的执行可以被完全中断。

实验五 线下验收表

序号	指令 PC=0	RV32 汇编语句	控制信号赋值, 未列出的控制信号值为 0	输出信号值
1	008000ef	jal x1, 8	ExtOp=4, RegWr=1, ALUBSrc=1, Branch=1	x1: 00000004
2	ffdf06f	jal x0, -4	ExtOp=4, RegWr=1, ALUBSrc=1, Branch=1	x0: 00000000
3	00100013	addi x0, x0, 1	RegWr=1, ALUBSrc=2	x0: 00000000
4	deadc2b7	lui x5, -136484	ExtOp=1, RegWr=1, ALUBSrc=2, ALUCtr=15	x5: deadc000
5	eef28293	addi x5, x5, -273	RegWr=1, ALUBSrc=2	x5: deadbeef
6	0052d333	srl x6, x5, x5	RegWr=1, ALUCtr=5	x6: 0001bd5b
7	00135313	srli x6, x6, 1	RegWr=1, ALUBSrc=2, ALUCtr=5	x6: 0000dead
8	4052d3b3	sra x7, x5, x5	RegWr=1, ALUCtr=13	x7: ffffbd5b
9	00529e33	sll x28, x5, x5	RegWr=1, ALUCtr=1	x28: df778000
10	001e1e13	slli x28, x28, 1	RegWr=1, ALUBSrc=2, ALUCtr=1	x28: beef0000
11	006e0eb3	add x29, x28, x6	RegWr=1	x29: beefdead
12	01000513	addi x10, x0, 16	RegWr=1, ALUBSrc=2	x10: 00000010
13	005500a3	sb x5, 1(x10)	ExtOp=2, ALUBSrc=2, MemWr=1, MemOp=5	M[0x10]: 0000ef00
14	00551223	sh x5, 4(x10)	ExtOp=2, ALUBSrc=2, MemWr=1, MemOp=6	M[0x14]: 0000beef
15	00551323	sh x5, 6(x10)	ExtOp=2, ALUBSrc=2, MemWr=1, MemOp=6	M[0x14]: beefbeef
16	00552423	sw x5, 8(x10)	ExtOp=2, ALUBSrc=2, MemWr=1	M[0x18]: deadbeef
17	00150483	lb x9, 1(x10)	RegWr=1, ALUBSrc=2, MemOp=5, MemToReg=1	x9: ffffffffef
18	00154903	lbu x18, 1(x10)	RegWr=1, ALUBSrc=2, MemOp=1, MemToReg=1	x18: 000000ef
19	00451983	lh x19, 4(x10)	RegWr=1, ALUBSrc=2, MemOp=6, MemToReg=1	x19: ffffbeef
20	00655a03	lhu x20, 6(x10)	RegWr=1, ALUBSrc=2, MemOp=2, MemToReg=1	x20: 0000beef
21	00452a83	lw x21, 4(x10)	RegWr=1, ALUBSrc=2, MemToReg=1	x21: beefbeef
22	00a55b03	lhu x22, 10(x10)	RegWr=1, ALUBSrc=2, MemOp=2, MemToReg=1	x22: 0000dead
23	ffff0bb7	lui x23, -16	ExtOp=1, RegWr=1, ALUBSrc=2, ALUCtr=15	x23: ffff0000
24	017afc33	and x24, x21, x23	RegWr=1, ALUCtr=7	x24: beef0000
25	016c6cb3	or x25, x24, x22	RegWr=1, ALUCtr=6	x25: beefdead
26	fb9e92e3	bne x29, x25, -92	ExtOp=3, ALUCtr=2, Branch=5	PC: 00000068
27	00000073	ecall		PC: 0000006c